LAPORAN RESMI

MODUL II

FOREIGN KEY DAN DML DASAR

PENGANTAR BASIS DATA

NAMA	**:**	**RAHMAD BAGUS SETIAWAN**
N.R.P	**:**	**250441100031**
DOSEN	**:**	**MASHUDAH SABILATURRIZQI, S.Kom., M.kom**
ASISTEN	**:**	**DESTYA NURFAIZA MUSLIM**
TGL PRAKTIKUM	**:**	**03 APRIL 2026**

Disetujui : ... APRIL 2026
Asisten

DESTYA NURFAIZA MUSLIM
24.04.411.00053

LABORATORIUM TEKNOLOGI INFORMASI

PRODI SISTEM INFORMASI

JURUSAN TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS TRUNODJOYO MADURA

DAFTAR ISI

DAFTAR ISI ..... **i**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan 1
BAB II DASAR TEORI **2**
2.1 Relasi Antar Tabel dalam Database 2
2.2 *Primary Key* 2
2.3 *Foreign Key* 3
2.4 Jenis Relasi dalam Database 3
2.5 Aturan Aksi pada *Foreign Key (Referential Action)* 4
2.5.1 *ON DELETE CASCADE* 4
2.5.2 ON DELETE RESTRICT 5
2.5.3 ON DELETE NULL 6
2.5.4 Perbandingan *ON DELETE* 7
2.6 Data Manipulation Language (DML) 7
2.7 Perintah INSERT 7
2.8 Perintah UPDATE 8
2.9 Perintah DELETE 8
2.10 Perintah TRUNCATE 8
BAB III TUGAS PENDAHULUAN **9**
BAB IV IMPLEMENTASI **12**
4.1 Tugas Praktikum 12
4.1.1 Tugas Praktikum Soal No. 1 12
4.1.2 Tugas Praktikum Soal No. 2 12
4.1.3 Tugas Praktikum Soal No. 3 12
4.1.4 Tugas Praktikum Soal No. 4 13
4.1.5 Tugas Praktikum Soal No. 5 13
4.2 Source Code 13
4.2.1 Source Code Soal No. 1 13
4.2.2 Source Code Soal No. 2 14
4.2.3 Source Code Soal No. 3 15
4.2.4 Source Code Soal No. 4 15
4.2.5 Source Code Soal No. 5 16

4.3 Hasil 16
4.3.1 Hasil Soal No. 1 16
4.3.2 Hasil Soal No. 2 17
4.3.3 Hasil Soal No. 3 18
4.3.4 Hasil Soal No. 4 18
4.3.5 Hasil Soal No. 5 19
4.4 Penjelasan 19
4.4.1 Penjelasan Soal No. 1 19
4.4.2 Penjelasan Soal No. 2 20
4.4.3 Penjelasan Soal No. 3 20
4.4.4 Penjelasan Soal No. 4 20
4.4.5 Penjelasan Soal No. 5 21
BAB V PENUTUP 22
5.1 Analisa 22
5.2 Kesimpulan 22

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem informasi yang semakin kompleks menuntut adanya pengelolaan data yang terstruktur dan saling terhubung dengan baik. Dalam basis data relasional, data tidak hanya disimpan dalam satu tabel, tetapi dibagi ke dalam beberapa tabel yang saling berelasi. Oleh karena itu, diperlukan mekanisme untuk menjaga hubungan antar tabel agar tetap konsisten dan tidak terjadi inkonsistensi data. Salah satu konsep penting dalam hal ini adalah penggunaan *Foreign Key* yang berfungsi sebagai penghubung antar tabel sekaligus menjaga integritas data.

Selain itu, pengelolaan data dalam database dilakukan menggunakan *Data Manipulation Language (DML)* seperti *INSERT, UPDATE, DELETE, dan TRUNCATE*. Perintah-perintah ini memungkinkan pengguna untuk menambah, mengubah, maupun menghapus data sesuai kebutuhan sistem. Namun, dalam penggunaannya, DML harus memperhatikan constraint seperti *foreign key* agar tidak menimbulkan kesalahan atau pelanggaran integritas data.

Melalui praktikum ini, praktikan diharapkan dapat memahami konsep *Foreign Key* dan DML dasar serta mampu mengimplementasikannya dalam sistem database. Praktikan juga dilatih untuk menganalisis dampak dari setiap perintah yang digunakan, sehingga dapat merancang dan mengelola basis data yang konsisten, aman, dan sesuai dengan kebutuhan aplikasi di dunia nyata.

1.2 Tujuan

- Memahami konsep relasi antar tabel dalam basis data relasional.
- Menjelaskan fungsi *Primary Key* dan *Foreign Key* dalam database.
- Membuat relasi antar tabel menggunakan *Foreign Key*.
- Memahami aturan *referential integrity* pada *database*.
- Memahami penggunaan *ON DELETE CASCADE, RESTRICT,* dan *SET NULL.*
- Menambahkan data ke dalam tabel menggunakan perintah *INSERT.*
- Mengubah data dalam tabel menggunakan perintah *UPDATE.*
- Menghapus data menggunakan perintah *DELETE.*
- Menghapus seluruh data dalam tabel menggunakan *TRUNCATE.*

BAB II
DASAR TEORI

2.1 Relasi Antar Tabel dalam Database

Dalam sistem basis data relasional, data sering disimpan dalam beberapa tabel yang saling berhubungan. Hubungan antar tabel ini disebut relasi (*relationship*).

Tujuan relasi antar tabel adalah:

- Menghindari duplikasi data.
- Meningkatkan konsistensi data.
- Mempermudah pengelolaan data

Contoh sederhana terdapat pada sistem penjualan.

Tabel Kategori

id_kategori	nama_kategori
1	Laptop
2	Smartphone
3	Aksesoris

Tabel Produk

id_produk	nama_produk	id_kategori
101	Laptop Asus	1
102	Laptop Lenovo	1
103	iPhone	2

Pada contoh tersebut kolom *id_kategori* pada tabel produk merujuk pada tabel kategori.

Relasi ini memungkinkan satu kategori memiliki banyak produk.

2.2 *Primary Key*

Primary Key adalah atribut yang digunakan untuk mengidentifikasi setiap *record* secara unik dalam suatu tabel.

Karakteristik *primary key*:

- Tidak boleh bernilai *NULL*
- Tidak boleh memiliki nilai yang sama
- Hanya satu primary key dalam satu tabel

Contoh:

```sql
CREATE TABLE
    kategori (
        id_kategori INT PRIMARY KEY,
        nama_kategori VARCHAR(50)
    );
```

Pada contoh tersebut, id_kategori menjadi primary key.

2.3 *Foreign Key*

Foreign Key adalah atribut dalam suatu tabel yang digunakan untuk menghubungkan tabel tersebut dengan tabel lain.

Foreign key merujuk pada atribut kunci (*key*) pada tabel lain (biasanya primary *key*).

Contoh:

```sql
FOREIGN KEY (id_kategori)
REFERENCES kategori (id_kategori)
```

Dengan adanya *foreign key*, database dapat memastikan bahwa :

- Data yang dimasukkan valid
- Hubungan antar tabel tetap konsisten
- Integritas data terjaga

2.4 Jenis Relasi dalam Database

Relasi antar tabel umumnya terdiri dari tiga jenis kardinalitas.

1. *One to One (1:1)*

   Satu data pada tabel pertama berhubungan dengan satu data pada tabel kedua. Contoh:

   - Tabel mahasiswa dan tabel kartu_mahasiswa

2. *One to Many (1:N)*

   Satu data pada tabel pertama dapat berhubungan dengan banyak data pada tabel kedua. Contoh :

   - Satu kategori memiliki banyak produk

3. *Many to Many (N:M)*

   Banyak data pada tabel pertama berhubungan dengan banyak data pada tabel kedua. Contoh :

   - 1 mahasiswa mengambil banyak kelas mata kuliah
   - 1 kelas mata kuliah diambil banyak mahasiswa

Relasi ini pasti membutuhkan tabel penghubung.

2.5 Aturan Aksi pada *Foreign Key (Referential Action)*

Dalam sistem basis data relasional, *Foreign Key* tidak hanya berfungsi untuk menghubungkan dua tabel, tetapi juga dapat mengatur bagaimana sistem bereaksi ketika terjadi perubahan data pada tabel induk (*parent table*).

Perubahan yang dimaksud biasanya berupa:

- Penghapusan data (DELETE).
- Perubahan nilai *Primary Key* (UPDATE)

Untuk mengatur perilaku tersebut, DBMS menyediakan beberapa aturan yang disebut *referential action*, seperti:

- ON DELETE CASCADE
- ON DELETE RESTRICT
- ON DELETE SET NULL

Aturan ini digunakan pada saat mendefinisikan *foreign key*.

Contoh sintaks umum:

```
FOREIGN KEY (kolom_fk)
REFERENCES nama_tabel(kolom_pk)
ON DELETE aksi
```

2.5.1 *ON DELETE CASCADE*

ON DELETE CASCADE berarti jika sebuah data pada tabel induk dihapus, maka seluruh data pada tabel anak yang terkait juga akan ikut terhapus secara otomatis.

Aturan ini sering digunakan ketika data pada tabel anak tidak boleh ada tanpa data induknya.

Contoh Kasus

Misalkan terdapat dua tabel

Tabel Kategori

id_kategori	nama_kategori
1	Laptop
2	Smartphone
3	Aksesoris

Tabel Produk

id_produk	nama_produk	id_kategori
101	Laptop Asus	1
102	Laptop Lenovo	1
103	iPhone	2

Relasi dibuat sebagai berikut :

```
CREATE TABLE
    produk (
        id_produk INT PRIMARY KEY,
        nama_produk VARCHAR(100),
        id_kategori INT,
        FOREIGN KEY (id_kategori) REFERENCES kategori
(id_kategori) ON DELETE CASCADE
    );
```

Jika data berikut dihapus:

```
DELETE FROM kategori
WHERE id_kategori = 1;
```

Maka :

- Kategori “Laptop” akan terhapus
- Seluruh produk dengan ide_kategori = 1 juga akan ikut terhapus

Hasil tabel produk menjadi :

id_produk	nama_produk	id_kategori
103	Samsung S10	2

2.5.2 ON DELETE RESTRICT

ON DELETE RESTRICT berarti data pada tabel induk tidak dapat dihapus jika masih digunakan oleh tabel anak.

Aturan ini digunakan untuk mencegah kehilangan data relasi secara tidak sengaja.

Contoh

Relasi tabel:

```
CREATE TABLE
    produk (
        id_produk INT PRIMARY KEY,
        nama_produk VARCHAR(100),
        id_kategori INT,
        FOREIGN KEY (id_kategori) REFERENCES kategori
(id_kategori) ON DELETE RESTRICT
```

```
);
```

Jika pengguna mencoba menjalankan perintah :

```
DELETE FROM kategori
WHERE id_kategori = 1;
```

Sedangkan masih ada produk yang menggunakan kategori tersebut, maka sistem akan menampilkan error dan proses penghapusan tidak akan dilakukan.

Contoh pesan error:

`Cannot delete or update a parent row:`

`a foreign key constraint fails`

Hal ini bertujuan untuk menjaga integritas data.

2.5.3 ON DELETE NULL

ON DELETE SET NULL berarti jika data pada tabel induk dihapus, maka nilai foreign key pada tabel anak akan diubah menjadi NULL.

Artinya data pada tabel anak tetap ada, tetapi hubungannya dengan tabel induk dihapus.

Syarat penting:

Kolom foreign key harus boleh bernilai NULL.

Contoh

Relasi tabel:

```
CREATE TABLE
    produk (
        id_produk INT PRIMARY KEY,
        nama_produk VARCHAR(100),
        id_kategori INT NULL,
        FOREIGN KEY (id_kategori) REFERENCES kategori
(id_kategori) ON DELETE SET NULL
    );
```

Jika pengguna mencoba menjalankan perintah :

```
DELETE FROM kategori
WHERE id_kategori = 1;
```

Maka isi tabel produk berubah menjadi:

id_produk	nama_produk	id_kategori
101	Laptop Asus	Null
102	Laptop Lenovo	Null

Data produk tetap ada, tetapi tidak lagi memiliki kategori.

2.5.4 Perbandingan *ON DELETE*

Aturan	Dampak
CASCADE	Data anak ikut terhapus
RESTRICT	Penghapusan data induk ditolak
SET NULL	Foreign key diubah menjadi NULL

Catatan:

Pemilihan aturan ON DELETE harus disesuaikan dengan kebutuhan sistem.

Contoh penerapan:

Sistem/Jenis Data	Aturan yang sering digunakan
Data penjualan	RESTRICT
Log data	SET NULL
Data detail transaksi	CASCADE

Hal ini bertujuan untuk menjaga konsistensi dan integritas data dalam database.

2.6 Data Manipulation Language (DML)

Data Manipulation Language (DML) adalah bagian dari SQL yang digunakan untuk mengelola data dalam tabel.

Beberapa perintah DML yang sering digunakan:

Perintah	Fungsi
INSERT	Menambahkan data
UPDATE	Mengubah data
DELETE	Menghapus data
TRUNCATE	Menghapus seluruh data

2.7 Perintah INSERT

Digunakan untuk menambahkan data ke dalam tabel.

Sintaks:

```
INSERT INTO nama_tabel (kolom1, kolom2)
VALUES (nilai1, nilai2);
```

Contoh :

```
INSERT INTO kategori VALUES
(1,'Laptop'),
(2,'Smartphone'),
(3,'Aksesoris');
```

2.8 Perintah UPDATE

Digunakan untuk memperbarui data dalam tabel.

Sintaks:

```
UPDATE nama_tabel
SET kolom = nilai
WHERE kondisi;
```

Contoh :

```
UPDATE produk
SET harga = 7300000
WHERE id_produk = 102;
```

2.9 Perintah DELETE

Digunakan untuk menghapus data dalam tabel.

Sintaks:

```
DELETE FROM nama_tabel
WHERE kondisi;
```

Contoh :

```
DELETE FROM produk
WHERE id_produk = 103;
```

2.10 Perintah TRUNCATE

Digunakan untuk menghapus seluruh data dalam tabel secara cepat.

```
TRUNCATE TABLE produk;
```

Perbedaan DELETE dan TRUNCATE

DELETE	TRUNCATE
Dapat menggunakan WHERE	Tidak dapat menggunakan WHERE
Menghapus sebagian data	Menghapus seluruh data
Lebih lambat	Lebih cepat

Dicky 09/26 09

BAB III
TUGAS PENDAHULUAN

3.1 SOAL

1. Perhatikan dua struktur tabel berikut:
   - Tabel pelanggan (id_pelanggan, nama)
   - Tabel pesanan (id_pesanan, id_pelanggan, tanggal_pesanan)

   Jelaskan mana yang menjadi Primary Key dan mana foreign key, serta apa fungsi masing-masing. Apa risiko yang bisa terjadi jika kolom id_pelanggan di tabel pesanan tidak diberi Constraint Foreign key? Berikan satu contoh konkret.
2. Jelaskan perbedaan antara relasi One-to-many (1:N) dan Many-to-Many (N:M). Berikan masing-masing satu contoh dari dunia nyata. Mengapa relasi Many-to-Many memerlukan tabel penghubung (Junction Table), sementara relasi One-to-many tidak?
3. Jelaskan apa yang terjadi pada data produk dalam skenario berbeda ketika sebuah kategori dihapus:
   - Jika menggunakan ON DELETE CASCADE
   - Jika menggunakan ON DELETE RESTRICT
   - Jika menggunakan ON DELETE SET NULL

   untuk konteks toko online, mana pilihan yang paling sesuai dan mengapa?
4. Diberikan tabel kategori (id_kategori, nama) berisi {id = 1, nama = "Elektronik"} dan tabel Produk (id_produk, nama_produk, id_kategori) yang memiliki foreign key merujuk ke kategori. Tuliskan perintah INSERT untuk menambahkan satu produk ke dalam tabel Produk, Kemudian jelaskan apa yang akan terjadi jika kita mencoba INSERT produk dengan id_kategori = 99 (kategori yang tidak ada. Mengapa sistem menolaknya?

Dedy 09/04 26

5. Jelaskan apa yang akan terjadi jika menjalankan perintah UPDATE untuk mengubah nilai Id_kategori sebuh produk menjadi kategori yang tidak ada, apakah perintah akan berhasil atau gagal? mengapa demikian?
6. Jelaskan perbedaan antara perintah DELETE dan TRUNCATE dalam konteks foreign key, Jika tabel kategori memiliki foreign key yang merujuk kepadanya, di mana antara keduanya yang akan mempertimbangkan Constraint FK saat dijalankan? mengapa?

3.2 JAWABAN

1. Primary key dan Foreign Key
   a. Primary key
      - Id_pelanggan di tabel pelanggan
      - Id_Pesanan di babel pesanan
      - Fungsi : sebagai identitas unik tiap data
   b. foreign key
      - Id_Pelanggan di tabel pesanan
      - fungsi : ngambil relasi di tabel lain
   c. Risiko jika tidak ada FK
      - Bisa masuk data ngawur
      - Data jadi tidak konsisten
   d. Contoh
      - Ada pesanan dengan Id_pelanggan = 99
      - Padahal di tabel pelanggan tidak ada Id 99 (tidak jelas punya siapa)
2. One-to-Many vs Many-to-Many
   - One-to-many (1:N)
     a. Satu data punya banyak relasi
     b. Contoh : 1 pelanggan mempunyai banyak pesanan
   - Many-to-many (N:M)
     a. banyak ke banyak
     b. Contoh: Mahasiswa bisa ambil banyak matkul dan

CS Dipindai dengan CamScanner

Deity 09/26/09

Satu mata kuliah bisa diambil banyak mahasiswa

3. ON DELETE
   - CASCADE : Jika kategori dihapus, semua produk ikut terhapus
   - Restrict : Gabisa hapus kategori kalau masih dipakai
   - SET NULL : Produk tetap ada, tapi id_kategori jadi NULL

   yang paling cocok untuk toko online adalah Restrict karena biar data produk aman dan ga ikut terhapus

4. INSERT dan Error FK
   - Contoh :

     INSERT INTO produk (id_produk, nama_produk, id_kategori)
     VALUES (1, 'Laptop', 1);
   - Jika pakai id_kategori = 99
     a. akan ditolak / error
     b. karena kategori 99 tidak ada
   - Alasan :

     fk menjaga supaya data tetap valid

5. UPDATE ke kategori yang tidak ada
   - Hasil = gagal
   - karena melanggar aturan foreign key dan Database menjaga data tetap nyambung

6. DELETE VS TRUNCATE
   a. DELETE
      - hapus data satu per satu
      - tetap ngecek foreign key
   b. TRUNCATE
      - hapus semua data langsung
      - biasanya tidak bisa dijalankan kalau ada foreig key
   c. yang mempertimbangkan fk :
      - DELETE : karena dia jalan per data dan ngecek relasi

CS Dipindai dengan CamScanner

BAB IV
IMPLEMENTASI

4.1 Tugas Praktikum

Studi Kasus: Sistem Informasi Klinik Sehat Bersama

Klinik Sehat Bersama ingin membuat database untuk mengelola data dokter, pasien, dan rekam medis. Struktur tabelnya sebagai berikut:

- dokter (id_dokter, nama_dokter, spesialisasi)
- pasien (id_pasien, nama_pasien, tanggal_lahir, no_telepon)
- rekam_medis (id_rekam, id_pasien, id_dokter, tanggal_periksa, diagnosis)

Tabel rekam_medis memiliki dua Foreign Key**:** id_pasien merujuk ke tabel pasien, dan id_dokter merujuk ke tabel dokter.

4.1.1 Tugas Praktikum Soal No. 1

Tuliskan perintah SQL untuk membuat ketiga tabel di atas. Tabel rekam_medis harus memiliki Foreign Key untuk id_pasien dan id_dokter dengan aturan ON DELETE RESTRICT. Jelaskan mengapa ON DELETE RESTRICT dipilih untuk tabel rekam_medis dalam konteks klinik ini!

4.1.2 Tugas Praktikum Soal No. 2

Setelah tabel terbentuk, lakukan langkah-langkah berikut

- Tambahkan minimal 2 data dokter, 2 data pasien, dan 2 data rekam medis yang valid ke dalam tabel masing-masing.
- Coba tambahkan satu rekam medis dengan id_dokter yang tidak ada di tabel dokter. Tuliskan querynya, sertakan pesan error yang muncul, dan jelaskan mengapa sistem menolaknya.

4.1.3 Tugas Praktikum Soal No. 3

Dengan data yang sudah ada dari Soal 2, lakukan perintah berikut dan analisis hasilnya:

- Ubah nomor telepon salah satu pasien menggunakan perintah UPDATE.
- Ubah spesialisasi salah satu dokter menggunakan perintah UPDATE.
- Hapus salah satu rekam medis menggunakan perintah DELETE dengan klausa WHERE. Sertakan screenshot sebelum dan sesudah penghapusan.

4.1.4 Tugas Praktikum Soal No. 4

Buat ulang tabel rekam_medis dengan ON DELETE CASCADE untuk Foreign Key id_dokter. FK id_pasien tetap RESTRICT. Lakukan langkah berikut:

- Sertakan query CREATE TABLE rekam_medis yang baru.
- Pastikan minimal ada 2 rekam medis yang menggunakan id_dokter yang sama, lalu hapus dokter tersebut. Tuliskan query DELETE-nya.
- Tampilkan isi tabel rekam_medis setelah penghapusan menggunakan SELECT. Sertakan screenshot hasilnya.
- Jelaskan apa yang terjadi dan bandingkan: bagaimana hasilnya jika tabel rekam_medis menggunakan ON DELETE RESTRICT atau ON DELETE SET NULL pada FK id_dokter?

4.1.5 Tugas Praktikum Soal No. 5

Buat ulang tabel rekam_medis dengan ON DELETE RESTRICT untuk kedua FK-nya, lalu isi kembali dengan minimal 2 data rekam medis. Kemudian lakukan langkah berikut:

- Jalankan TRUNCATE TABLE rekam_medis, lalu coba jalankan TRUNCATE TABLE pasien. Jalankan kedua query, sertakan screenshot, dan jelaskan hasilnya
- Jelaskan perbedaan DELETE dan TRUNCATE berdasarkan empat aspek berikut: penggunaan WHERE, kecepatan, pengaruh terhadap Foreign Key constraint, dan apakah bisa di-rollback dalam sebuah transaksi.

4.2 Source Code

4.2.1 Source Code Soal No. 1

```
CREATE TABLE
    dokter (
        id_dokter BIGINT PRIMARY KEY,
        nama_dokter VARCHAR(250),
        spesialisasi VARCHAR(250)
    );

CREATE TABLE
    pasien (
        id_pasien BIGINT PRIMARY KEY,
        nama_pasien VARCHAR(250),
        tanggal_lahir DATE,
        no_telepon CHAR(12)
```

```
    );

CREATE TABLE
    rekam_medis (
        id_rekam BIGINT PRIMARY KEY,
        id_pasien BIGINT,
        id_dokter BIGINT,
        tanggal_periksa DATE,
        diagnosis VARCHAR(250),
        FOREIGN KEY (id_pasien) REFERENCES pasien (id_pasien) ON
DELETE RESTRICT,
        FOREIGN KEY (id_dokter) REFERENCES dokter (id_dokter) ON
DELETE RESTRICT
    );
```

4.2.2 Source Code Soal No. 2

```
INSERT INTO
    dokter (id_dokter, nama_dokter, spesialisasi)
VALUES
    (1, 'Rahmad Bagus Setiawan', 'Bedah Jantung'),
    (2, 'Miftahul Ulum', 'Bedah Otak');

INSERT INTO
    pasien (id_pasien, nama_pasien, tanggal_lahir, no_telepon)
VALUES
    (1, 'Tyoo', '2006-12-18', '081234567890'),
    (2, 'Jhony', '2002-10-19', '080987654321');

INSERT INTO
    rekam_medis (
        id_rekam,
        id_pasien,
        id_dokter,
        tanggal_periksa,
        diagnosis
    )
VALUES
    (1, 2, 2, '2026-04-09', 'pusing'),
    (2, 1, 1, '2026-04-05', 'sakit hati');

INSERT INTO
    rekam_medis (
        id_rekam,
        id_pasien,
        id_dokter,
        tanggal_periksa,
```

```
        diagnosis
    )
VALUES
    (1, 1, 5, '2026-04-09', 'demam tinggi');
```

4.2.3 Source Code Soal No. 3

```
-- NOMOR 3
-- a
UPDATE pasien
SET no_telepon = '088888888888'
WHERE   id_pasien = 1;

-- b
UPDATE dokter
SET spesialisasi = 'Potong Kepala'
WHERE   id_dokter = 2;

-- c
DELETE FROM rekam_medis
WHERE id_rekam = 1;
```

4.2.4 Source Code Soal No. 4

```
CREATE TABLE
    rekam_medis (
        id_rekam BIGINT PRIMARY KEY,
        id_pasien BIGINT,
        id_dokter BIGINT,
        tanggal_periksa DATE,
        diagnosis VARCHAR(250),
        FOREIGN KEY (id_pasien) REFERENCES pasien (id_pasien) ON
DELETE RESTRICT,
        FOREIGN KEY (id_dokter) REFERENCES dokter (id_dokter) ON
DELETE CASCADE
    );

INSERT INTO
    rekam_medis (
        id_rekam,
        id_pasien,
        id_dokter,
        tanggal_periksa,
        diagnosis
    )
VALUES
    (1, 1, 1, '2026-04-02', 'SAKIT HATI'),
```

```
    (2, 2, 1, '2026-04-05', 'ANEMIA'),
    (3, 2, 2, '2026-04-05', 'SAKIT DEMAM'),
    (4, 1, 2, '2026-04-09', 'PUSING');


DELETE FROM dokter WHERE id_dokter = 2;
DELETE FROM pasien WHERE id_pasien = 1;
```

4.2.5 Source Code Soal No. 5

```
CREATE TABLE
    rekam_medis (
        id_rekam BIGINT PRIMARY KEY,
        id_pasien BIGINT,
        id_dokter BIGINT,
        tanggal_periksa DATE,
        diagnosis VARCHAR(250),
        FOREIGN KEY (id_pasien) REFERENCES pasien (id_pasien) ON
DELETE RESTRICT,
        FOREIGN KEY (id_dokter) REFERENCES dokter (id_dokter) ON
DELETE RESTRICT
    );

INSERT INTO
    rekam_medis (
        id_rekam,
        id_pasien,
        id_dokter,
        tanggal_periksa,
        diagnosis
    )
VALUES
    (1, 1, 1, '2026-04-02', 'DBD'),
    (2, 2, 1, '2026-04-05', 'usus buntu');

TRUNCATE TABLE rekam_medis;

TRUNCATE TABLE pasien;
```

4.3 Hasil

4.3.1 Hasil Soal No. 1

Tabel Dokter

#	Nama	Jenis	Penyortiran	Atribut	Tak Ternilai	Bawaan	Komentar	Ekstra	Tindakan
1	**id_dokter**	bigint(20)			Tidak	*Tidak ada*			Ubah Hapus Lainnya
2	**nama_dokter**	varchar(250)	utf8mb4_general_ci		Ya	*NULL*			Ubah Hapus Lainnya
3	**spesialisasi**	varchar(250)	utf8mb4_general_ci		Ya	*NULL*			Ubah Hapus Lainnya

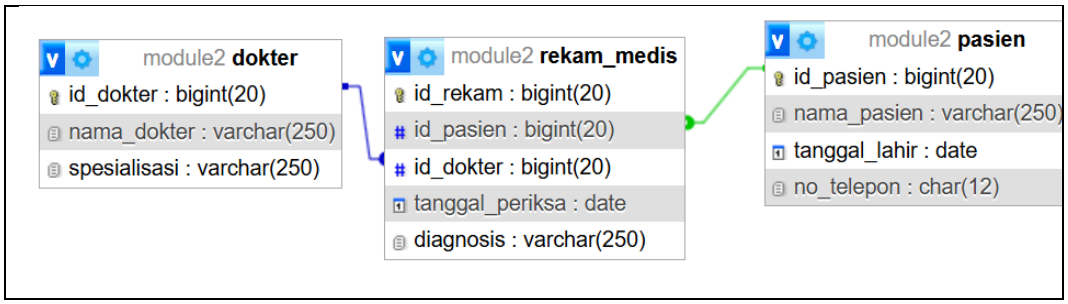
Tabel Pasien

#	Nama	Jenis	Penyortiran	Atribut	Tak Ternilai	Bawaan	Komentar	Ekstra	Tindakan
1	**id_pasien**	bigint(20)			Tidak	*Tidak ada*			Ubah Hapus Lainnya
2	**nama_pasien**	varchar(250)	utf8mb4_general_ci		Ya	*NULL*			Ubah Hapus Lainnya
3	**tanggal_lahir**	date			Ya	*NULL*			Ubah Hapus Lainnya
4	**no_telepon**	char(12)	utf8mb4_general_ci		Ya	*NULL*			Ubah Hapus Lainnya

Tabel Rekam_Medis

#	Nama	Jenis	Penyortiran	Atribut	Tak Ternilai	Bawaan	Komentar	Ekstra	Tindakan
1	**id_rekam**	bigint(20)			Tidak	*Tidak ada*			Ubah Hapus Lainnya
2	**id_pasien**	bigint(20)			Ya	*NULL*			Ubah Hapus Lainnya
3	**id_dokter**	bigint(20)			Ya	*NULL*			Ubah Hapus Lainnya
4	**tanggal_periksa**	date			Ya	*NULL*			Ubah Hapus Lainnya
5	**diagnosis**	varchar(250)	utf8mb4_general_ci		Ya	*NULL*			Ubah Hapus Lainnya

Hubungan



4.3.2 Hasil Soal No. 2

Tabel Dokter

	id_dokter	nama_dokter	spesialisasi
Ubah Salin Hapus	1	Rahmad Bagus Setiawan	Bedah Jantung
Ubah Salin Hapus	2	Miftahul Ulum	Bedah Otak

Tabel Pasien

	id_pasien	nama_pasien	tanggal_lahir	no_telepon
Ubah Salin Hapus	1	Tyoo	2006-12-18	081234567890
Ubah Salin Hapus	2	Jhony	2002-10-19	080987654321

Tabel Rekam_Medis

	id_rekam	id_pasien	id_dokter	tanggal_periksa	diagnosis
Ubah Salin Hapus	1	2	2	2026-04-09	pusing
Ubah Salin Hapus	2	1	1	2026-04-05	sakit hati

Pesan Error

```
INSERT INTO
    rekam_medis (
        id_rekam,
        id_pasien,
        id_dokter,
        tanggal_periksa,
```

MySQL menyatakan:

```
#1452 - Cannot add or update a child row: a foreign key constraint fails (`module2`.`rekam_medis`, CONSTRAINT `rekam_medis_ibfk_2`
FOREIGN KEY (`id_dokter`) REFERENCES `dokter` (`id_dokter`))
```

4.3.3 Hasil Soal No. 3

Tabel Dokter

			id_dokter	nama_dokter	spesialisasi
Ubah	Salin	Hapus	1	Rahmad Bagus Setiawan	Bedah Jantung
Ubah	Salin	Hapus	2	Miftahul Ulum	Potong Kepala

Tabel Pasien

			id_pasien	nama_pasien	tanggal_lahir	no_telepon
Ubah	Salin	Hapus	1	Tyoo	2006-12-18	081234567890
Ubah	Salin	Hapus	2	Jhony	2002-10-19	088888888888

Tabel Rekam_Medis

Sebelum dihapus :

			id_rekam	id_pasien	id_dokter	tanggal_periksa	diagnosis
Ubah	Salin	Hapus	1	2	2	2026-04-09	pusing
Ubah	Salin	Hapus	2	1	1	2026-04-05	sakit hati

Sesudah dihapus :

			id_rekam	id_pasien	id_dokter	tanggal_periksa	diagnosis
Ubah	Salin	Hapus	2	1	1	2026-04-05	sakit hati

4.3.4 Hasil Soal No. 4

Tabel Rekam_MEdis

Sebelum Kolom id_dokter dihapus :

			id_rekam	id_pasien	id_dokter	tanggal_periksa	diagnosis
Ubah	Salin	Hapus	1	1	1	2026-04-02	SAKIT HATI
Ubah	Salin	Hapus	2	2	1	2026-04-05	ANEMIA
Ubah	Salin	Hapus	3	2	2	2026-04-05	SAKIT DEMA
Ubah	Salin	Hapus	4	1	2	2026-04-09	PUSING

Sesudah Sebelum Kolom id_dokter dihapus (CASCADE):

			id_rekam	id_pasien	id_dokter	tanggal_periksa	diagnosis
Ubah	Salin	Hapus	1	1	1	2026-04-02	SAKIT HAT
Ubah	Salin	Hapus	2	2	1	2026-04-05	ANEMIA

Sesudah Sebelum Kolom id_dokter dihapus (CASCADE):

```
DELETE FROM pasien WHERE id_pasien = 1;

lySQL menyatakan:

1451 - Cannot delete or update a parent row: a foreign key constraint fails (`module2`.`rekam_medis`, CONSTRAINT `rekam_medis_ibfk
OREIGN KEY (`id_pasien`) REFERENCES `pasien` (`id_pasien`))
```

4.3.5 Hasil Soal No. 5

Tabel rekam_medis :

			id_rekam	id_pasien	id_dokter	tanggal_periksa	diagnosis
Ubah	Salin	Hapus	1	1	1	2026-04-02	DBD
Ubah	Salin	Hapus	2	2	1	2026-04-05	usus buntu

id_rekam	id_pasien	id_dokter	tanggal_periksa	diagnosis

```
MySQL menyatakan:

#1701 - Cannot truncate a table referenced in a foreign key constraint (`module2`.`rekam_medis`, CONSTRAINT `rekam_medis_ibfk_1` FOREIGN
KEY (`id_pasien`) REFERENCES `module2`.`pasien` (`id_pasien`))
```

4.4 Penjelasan

4.4.1 Penjelasan Soal No. 1

Tugas praktikum nomor 1 menjelaskan tentang perancangan skema tabel pada sistem informasi klinik dengan menerapkan relasi antar tabel menggunakan foreign key. Dalam praktikum ini dibuat tiga tabel yaitu dokter, pasien, dan rekam_medis, di mana tabel rekam_medis memiliki foreign key id_pasien dan id_dokter yang merujuk ke tabel pasien dan dokter dengan aturan ON DELETE RESTRICT.

Kenapa Menggunakan ON DELETE RESTRICT karena dalam konteks klinik karena data rekam medis bersifat krusial dan harus tetap tersimpan sebagai riwayat pemeriksaan pasien, sehingga tidak boleh hilang hanya karena data induknya dihapus, dengan menggunakan ON DELETE RESTRICT untuk menjaga

integritas data agar jika ada hal yang tidak diinginkan seperti tidak sengaja terhapusnya data.

4.4.2 Penjelasan Soal No. 2

Tugas praktikum nomor 2 menjelaskan tentang proses penambahan data ke dalam tabel yang telah memiliki relasi serta pengujian integritas data menggunakan foreign key. Pada praktikum ini, dilakukan penambahan data ke tabel dokter, pasien, dan rekam_medis dengan data yang valid sehingga proses INSERT berhasil karena nilai foreign key sesuai dengan data pada tabel induknya.

Kenapa bisa error, dikarenakan nilai foreign key harus merujuk pada data yang valid di tabel induk. Hal ini menunjukkan bahwa foreign key berfungsi untuk menjaga konsistensi dan keakuratan data, sehingga tidak memungkinkan adanya data yang tidak memiliki hubungan yang jelas dalam database.

4.4.3 Penjelasan Soal No. 3

Tugas praktikum nomor 3 menjelaskan tentang manipulasi data menggunakan perintah Data Manipulation Language (DML) yaitu UPDATE dan DELETE pada tabel yang sudah memiliki relasi. Pada praktikum ini dilakukan perubahan data nomor telepon pada tabel pasien dan perubahan spesialisasi pada tabel dokter menggunakan perintah UPDATE, sehingga data yang dipilih berdasarkan kondisi tertentu dapat diperbarui tanpa mempengaruhi data lainnya.

4.4.4 Penjelasan Soal No. 4

Tugas praktikum nomor 4 menjelaskan tentang penerapan aturan ON DELETE CASCADE pada foreign key serta dampaknya terhadap relasi antar tabel. Pada praktikum ini, tabel rekam_medis dibuat ulang dengan foreign key id_dokter menggunakan ON DELETE CASCADE, sedangkan id_pasien tetap menggunakan ON DELETE RESTRICT. Kemudian ditambahkan data rekam medis yang menggunakan id_dokter yang sama, dan dilakukan penghapusan data dokter tersebut.

Jika tabel rekam_medis dengan ON DELETE RESTRICT pada FK id_dokter, jika ada penghapusan data pada tabel Dokter maka system akan menolak perintah tersebut. Jika ON DELETE SET NULL pada FK id_dokter, jika ada penghapusan data pada tabel Dokter system akan mengizinkan perintah itu, namun pada tabel rekam_medis isi data pada kolom id_dokter berisi “NULL”.

4.4.5 Penjelasan Soal No. 5

Tugas praktikum nomor 5 menjelaskan tentang pengaruh penggunaan perintah TRUNCATE dalam tabel yang memiliki relasi foreign key serta perbandingannya dengan perintah DELETE. Pada praktikum ini, tabel rekam_medis dibuat ulang dengan kedua foreign key menggunakan ON DELETE RESTRICT, kemudian dilakukan TRUNCATE TABLE rekam_medis yang berhasil karena tidak melanggar relasi. Namun, ketika menjalankan TRUNCATE TABLE pasien, sistem akan menolak perintah tersebut karena tabel pasien masih direferensikan oleh tabel rekam_medis melalui foreign key, sehingga integritas data tetap terjaga.

BAB V
PENUTUP

5.1 Analisa

Dari hasil praktikum, praktikan menganalisis bahwa Foreign Key dan DML dasar memiliki peran penting dalam menjaga hubungan antar tabel serta pengelolaan data dalam database. Foreign key digunakan untuk menghubungkan tabel satu dengan yang lain sehingga dapat menjaga konsistensi dan integritas data, sedangkan aturan seperti ON DELETE CASCADE, RESTRICT, dan SET NULL menentukan bagaimana sistem merespon perubahan atau penghapusan data pada tabel induk. Oleh karena itu, penggunaan DML harus dilakukan dengan hati-hati agar tidak melanggar constraint yang ada. Secara keseluruhan, praktikan menyimpulkan bahwa pemahaman Foreign Key dan DML sangat penting untuk membangun sistem database yang konsisten, aman, dan sesuai dengan kebutuhan aplikasi.

5.2 Kesimpulan

Dari hasil praktikum, praktikan menyimpulkan bahwa Foreign Key dan DML dasar merupakan komponen penting dalam sistem basis data relasional yang berfungsi untuk menjaga hubungan antar tabel serta mengelola data secara terstruktur. Foreign key memastikan integritas dan konsistensi data antar tabel, sedangkan perintah DML digunakan untuk melakukan manipulasi data seperti menambah, mengubah, dan menghapus data sesuai kebutuhan sistem. Pemahaman keduanya sangat penting agar database dapat berjalan dengan aman, konsisten, dan sesuai dengan aturan yang telah ditetapkan.

a) Foreign Key digunakan untuk menghubungkan tabel dan menjaga integritas data
b) Primary Key sebagai identitas unik yang menjadi acuan Foreign Key
c) Aturan ON DELETE (CASCADE, RESTRICT, SET NULL) menentukan aksi terhadap data relasi.
d) Foreign Key dapat menyebabkan error jika data yang dimasukkan atau dihapus melanggar relasi
e) Pemilihan aturan relasi dan penggunaan DML harus disesuaikan dengan kebutuhan sistem